

method is compared in Figure 4. The test file used is publicly available at <https://speed.hetzner.de/10MB.bin>.

Sequential approach

(synchronous downloading): In a sequential (or synchronous) approach, each file download begins only after the previous one has completed as shown in Figure 2. This method is simple to implement and easy to understand, making it a common choice for beginners. However, it has a significant downside when dealing with I/O-bound tasks like network requests — it leads to inefficient use of time and system resources. The total time taken becomes the sum of all individual wait times, which can be substantial when downloading multiple large files. Figure 2 illustrates this approach using Python's requests library.

Asynchronous approach

(using asyncio + aiohttp): In contrast to the sequential method, the asynchronous approach allows multiple file downloads to proceed concurrently without waiting for each one to finish before starting the next as shown in Figure 3. Python's asyncio framework, combined with the aiohttp library, provides a powerful and efficient way to perform non-blocking HTTP requests. By leveraging the event loop and coroutines, this approach overlaps the waiting time for network responses, significantly reducing the total execution time. Figure 3 demonstrates how asynchronous programming enables scalable and time-efficient file downloading in Python.

Measuring the benefits of asynchronous programming

To clearly observe the performance difference, the same 10MB file was downloaded 10 times using both sequential and asynchronous approaches. The sequential method took approximately 11.96 seconds, as each download was processed

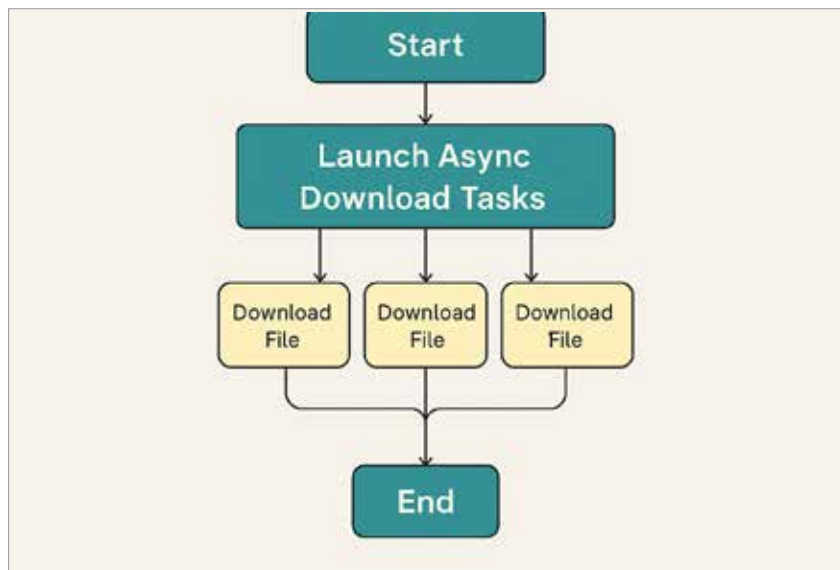


Figure 1: Block diagram showing parallel async download tasks

```

import requests
import time
import urllib3
urllib3.disable_warnings(urllib3.exceptions.InsecureRequestWarning)

URL = "https://speed.hetzner.de/10MB.bin"

def download_file(i):
    print(f"Downloading file {i}...")
    response = requests.get(URL, verify=False) # Disable SSL verification
    print(f"Downloaded file {i}: {len(response.content)} bytes")

start = time.time()
for i in range(10):
    download_file(i)
end = time.time()

print(f"Sequential downloading took {end - start:.2f} seconds")
  
```

```

Downloading file 0...
Downloaded file 0: 1059 bytes
Downloading file 1...
Downloaded file 1: 1059 bytes
Downloading file 2...
Downloaded file 2: 1059 bytes
Downloading file 3...
Downloaded file 3: 1059 bytes
Downloading file 4...
Downloaded file 4: 1059 bytes
Downloading file 5...
Downloaded file 5: 1059 bytes
Downloading file 6...
Downloaded file 6: 1059 bytes
Downloading file 7...
Downloaded file 7: 1059 bytes
Downloading file 8...
Downloaded file 8: 1059 bytes
Downloading file 9...
Downloaded file 9: 1059 bytes
Sequential downloading took 11.96 seconds
  
```

Figure 2: Sequential download output and timing